Synthesizing Multi-Agent Harnesses for Vulnerability Discovery

Hanzhi Liu Chaofan Shou Hongbo Wen
Yanju Chen Ryan Jingyang Fang Yu Feng

UC Santa Barbara · UC San Diego · Fuzzland · World Liberty Financial

arxiv.org/abs/2604.20801

LLM Agents Are Finding Real Bugs Now

Google Big Sleep

Zero-day **memory-corruption** bug
in SQLite (`CVE-2025-6965`).

Found by an LLM agent
before attackers exploited it.

Anthropic Mythos Preview

27-year-old DoS in OpenBSD.
Crashes a host with **2 packets**.

Found by an LLM agent
after decades of fuzzers missed it.

Also: agents autonomously hacked real websites; reproduced CTF exploits from natural language; teams of agents outperformed solos on zero-day discovery.

The obvious question: **how are these things actually built**
— and why do some work so much better than others?

What Is an "LLM Agent"? — No-Jargon Definition

agent = LLM + system prompt + tools

LLM
a language model
(Claude, GPT, Kimi, ...)
reads text, produces text

System prompt
NL instruction:
"find a memory-safety bug in this TIFF parser"

Tools
concrete actions each step:
`read_file`, `compile`, `run_binary`, `grep`, ...

Execution loop:

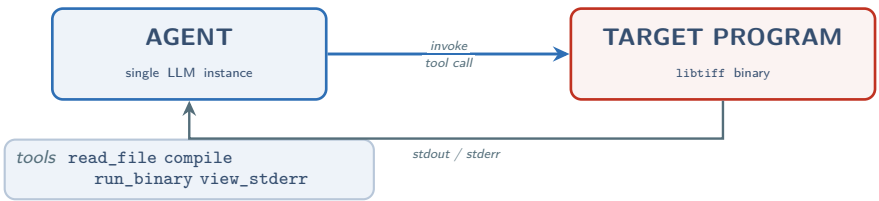


The **model** decides which tool to call next — not the human.

A Concrete Single-Agent Vulnerability Finder

System prompt: **"Find a memory-safety bug in libtiff's TIFF parser."**

Tools: `read_file`, `compile`, `run_binary`, `view_stderr`



One iteration: read source → craft input → run target → read output → repeat.

A **real** working architecture — and it has a **ceiling**.

Why a Single Agent Hits a Ceiling

① Context explosion

One instrumented Chrome build emits **megabytes** of sanitizer + coverage data **per run**. Frontier context windows top out at ∼200K tokens. **One run blows out the entire context.**

② Lost-in-the-middle

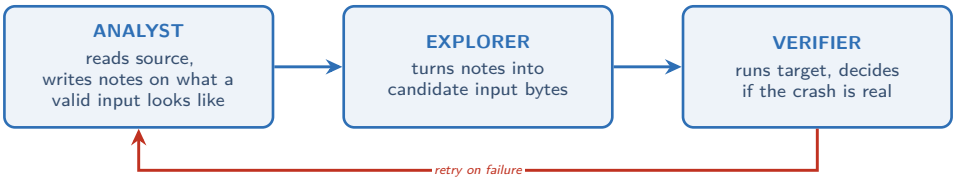
Source analysis, input crafting, crash triage **share the same context**. The model forgets earlier analysis, redoes work, and abandons long-horizon strategies.

③ Single thread of execution

One trace = one strategy at a time. A dead end wastes the **entire budget** — no parallel exploration.

Production systems don't use one agent. They use **teams**.

From a Soloist to a Team



*Each agent has its own prompt, its own tools, possibly its own model. Each one's context contains **only what it needs**.*

A team of three is strictly more powerful than a soloist — **but it introduces a new responsibility:**

▶ **who** exists? ▶ **who talks to whom?** ▶ what passes along each edge? ▶ when do we retry?

Someone has to **answer** those questions.

Meet the “Harness”

HARNESS — the **program** that decides:
▶ which agents exist ▶ what prompt each one receives ▶ which tools each one may call
▶ how outputs flow between agents ▶ how the team retries on failure

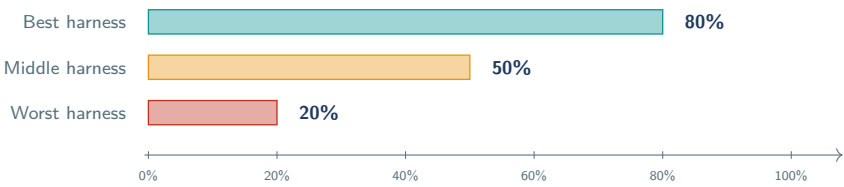
Analogy: **chips : circuit board :: agents (LLMs) : HARNESS**

Same chips, different board → totally different product. Same model, different harness → totally different capability.

In the wild: Big Sleep, Mythos, ForgeCode are all **harnesses** built on top of similar frontier models.

Why Harness Design Actually Matters

*Three TerminalBench-2 systems, **same** Claude Opus 4.6, same 89 tasks . . .*

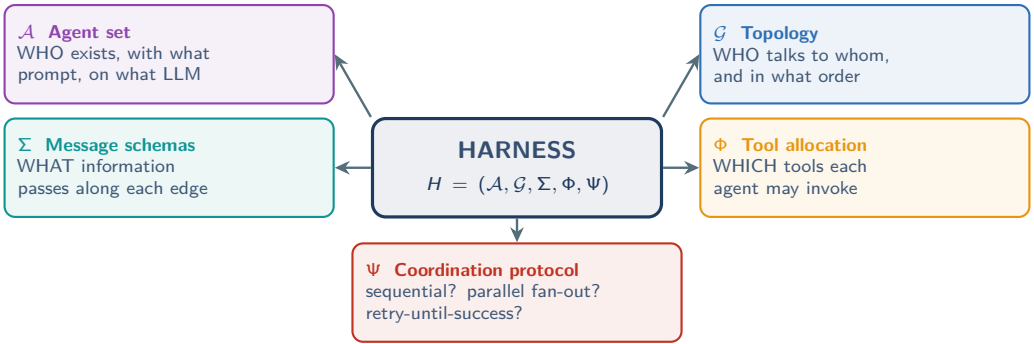


***4× spread** on the same model and tasks.*

Cost angle. Campaigns at this scale run into **tens of thousands of dollars** in inference. Good harness → more tasks, fewer LLM calls. Bad harness → burns the budget on dead-ends.

Picking the right harness is **not a tuning detail**. It's the **dominant lever**.

A Harness Has Five Knobs



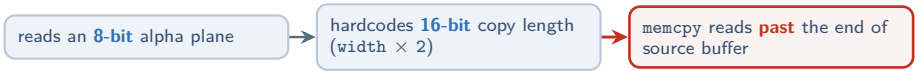
Designing a harness = choosing values for **all five knobs, jointly**.

(We'll refer back to $\mathcal{A}$ $\mathcal{G}$ Σ Φ Ψ *for the rest of the talk.)*

The Running Example: A Bug in `libheif`

Target: `libheif` — reference impl. of HEIF / HEVC images *(macOS Photos, iPhone backups, many web apps)*.

Bug: `CVE-2020-23109` — heap-buffer-overflow in color-conversion logic.



Trigger conditions (both must hold):

- ① the file must pass the parser's early **format-constraint check**
- ② the image must declare an **8-bit alpha channel**

Why this bug? ▶ too structured for random fuzzing ▶ two-condition trigger needs **real reasoning** ▶ real CVE, not a toy

Iteration 1: The File Gets Thrown Out

Harness: one generic agent **Prompt:** "Find a memory-safety bug in the HEIF parsing module."



⇒ *Parser threw the file out* ***before the vulnerable code ever ran****.*

Agent's own trace:
"I tested the parser."

Program's reality:
parser rejected the file in pre-flight.

A good harness should believe **the program**, **not the agent**.

Iteration 2: Right Function, Wrong Branch

Harness evolves: ***one agent*** → ***two agents***.



NEW signal: `cov` **line coverage**

```
color_convert()
  ↪ handle_8bit_alpha()
```

REACHED ✓
NEVER taken ×

The right function ran . . . but the ***buggy branch inside it*** *did not.*

Coverage tells us **exactly where we got stuck**. Pass/fail couldn't have told us this.

Iteration 3: Seed + Retry + Sanitizer

① **Start from a real file.** Take an existing HEIF on disk. Flip **one field** (alpha bit-depth). Leave every other byte alone.

② **Loop until something complains.** flip → run → look → adjust → **retry** until the binary reports an error.

③ **Wire in AddressSanitizer (san).** Catches out-of-bounds reads **even without a visible crash**.



Within a handful of tweaks: san reports a **heap-buffer-overflow READ** at `color_convert()`'s `memcpy`.

CVE-2020-23109 ✓ REPRODUCED

What Changed Across Those Three Iterations?

Looking at the five knobs from slide 9 . . .

$\mathcal{A}$ added agents: analyzer, crafter, verifier

$\mathcal{G}$ new edges: `analyzer` $\rightarrow$ `crafter` $\rightarrow$ `verifier` ; retry edge back

Σ verifier prompt: *"start from real file"*, *"loop until error"*

Φ added coverage-instrumented binary; added **san**

Ψ sequential $\rightarrow$ sequential + retry-until-success

All five knobs got touched. That is what good harness design **is**.
Today, every one of these edits was done **by hand**.

The question driving the rest of the talk:
Can a **machine** do this — automatically — on a new target?

Prior Work Each Covers a Slice

*Recall $H = (\mathcal{A}, \mathcal{G}, \Sigma, \Phi, \Psi)$. Each prior system searches **at most two** knobs.*

	$\mathcal{A}$	$\mathcal{G}$	Σ	Φ	Ψ
Meta-Harness	fixed	empty	•	•	fixed
ADAS	•	fixed	fixed	fixed	fixed
AFlow	fixed	•	fixed	fixed	•
MaAS	•	fixed	fixed	fixed	fixed

• = searched *fixed* = pinned by the tool's design

Each is a **real contribution**. But each **had to** fix the others to keep search tractable.

Two Fundamental Limitations

① NARROW SCOPE

No prior optimizer can in a **single step**:

add an agent AND rewire the graph
AND adjust the new agent's prompt
AND restrict its tool set.

But slide 14 just showed: that is **exactly** what the libheif story needed.

② COARSE FEEDBACK

Most optimizers use **binary pass/fail** outcomes:

"did the trial succeed?" → yes / no

Same number for: input never reached the bug · crash masked by an assertion · target's defense never exercised · ...

In a large search space, binary feedback turns the optimizer into a **random walk**.

These two limitations motivate the **two halves** of AGENTFLOW.

Insight 1 (vs. Narrow Scope): A Typed Graph DSL

A **harness** is a program in a typed graph DSL.

- **nodes** = agents
- **edges** = dataflow / retry links
- all five knobs ($\mathcal{A}$ $\mathcal{G}$ Σ Φ Ψ) = first-class **editable fields** in the same program

$\mathcal{A}$ AGENT SET $\Longleftarrow$ `agent(role, prompt, model, tools)`

$\mathcal{G}$ TOPOLOGY $\Longleftarrow$ edges: `a -> b`, `a.on_fail » b` (retry)

Σ MESSAGE SCHEMAS $\Longleftarrow$ template variables: `{{ analyst.out }}`

Φ TOOLS $\Longleftarrow$ per-agent tool-set field

Ψ COORDINATION $\Longleftarrow$ `fanout(n, k)`, guarded edges, graph topology

A single optimization step = a **local rewrite** of one program
$\rightarrow$ can add an agent + rewire + adjust prompt + restrict tools, **all at once**.

The DSL by Example

```
analyst = agent(role="analyst",
  prompt="Read source. Coverage={{ cov }}", tools={read_file, grep})
explorer = agent(role="explorer",
  prompt="Given {{ analyst.out }}, target branch {{ branch }}",
  tools={write_file, run_binary})
probes = fanout(explorer, 8)   # 8 parallel copies
validator = agent(role="validator",
  prompt="Probes: {{ probes.out }}. Sanitizer: {{ san }}. Pick the real crash.",
  tools={run_binary})
analyst -> probes   probes -> validator   validator.on_fail » analyst
```

Three things to notice: ▶ **nodes** are agents ▶ **edges** are dataflow or retry ▶ **feedback channels** reach an agent **only if** its prompt references them

The Type System as a Budget Guard

The proposer is itself an LLM. Its outputs are routinely broken: deleted agents still referenced · template vars referring to nothing · disconnected components.

Before any candidate is dispatched, the type checker asks three plain-English questions:

① Does every prompt reference only variables that will **actually be bound** at runtime?

② Does every declared edge feed information the downstream prompt **really uses**?

③ Is every node reachable from some source?

Linear-time graph traversal. ***Zero LLM calls.***

In practice ∼**20%** of proposer outputs **fail this check** → rejected before consuming any LLM inference.
Types as a budget guard.

Insight 2 (vs. Coarse Feedback): Four Runtime Channels

*Replace binary pass/fail with **four structured signals**, read from the target itself.*

① **test** Did the target's own test suite pass on the agent's input?

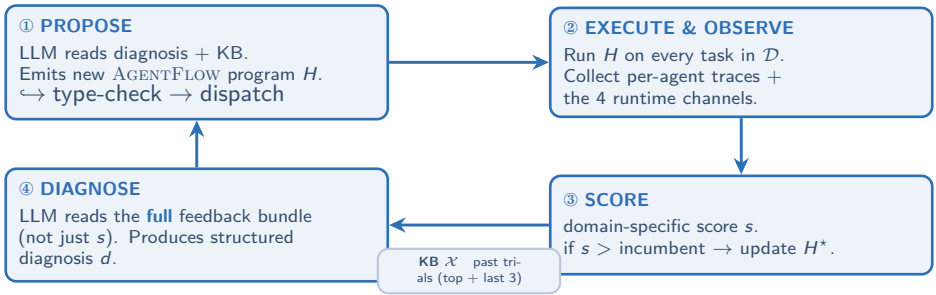
② **stderr / stdout** Raw text the target printed — errors, warnings.
("format constraint not satisfied" — slide 11)

③ **cov (line & branch coverage)** Which source lines / branches actually executed.
Reveals whether the input ever reached the buggy code. *("right fn, wrong branch" — slide 12)*

④ **san (AddressSanitizer / UBSan)** Catches memory-safety violations **even without a visible crash**. *(the bug-trigger signal — slide 13)*

Pass/fail tells you **that** it failed. These channels tell you **why** — and **which agent to fix**.

The AgentFlow Optimization Loop



Loop for K steps or until progress plateaus. **The diagnose box** is where rich feedback pays off.

Diagnosis: Four Structured Fields

*For each failed task, **one LLM call** produces:*

① **BOTTLENECK AGENT** Which agent most directly caused the failure?

② **INTENDED BEHAVIOR** What did that agent **think** it was doing? *(source: agent's own trace)*

③ **ACTUAL EXECUTION** What did the target program **actually** do? *(source: **cov** **san** **stderr**)*

④ **CORRECTIVE EDIT** What harness change closes the gap?

Crucial property: corrective edits are **HARNESS-DIRECTED**.

✓ *"rewrite the analyst's prompt to include precondition info"* ✓ *"route **san** to verifier, not analyst"*

× never: *"edit the target's source"* × never: *"rewrite the test inputs"*

Replay: AgentFlow Walks Through `libheif`

*Same bug as slides 11–13 — but now **nobody touches the harness by hand**.*

ITER 1 stderr : *"format constraint not satisfied"*

Diagnose: bottleneck = single generic agent; corrective = split into analyst + crafter
Propose: add `analyst` node, add `crafter` node, wire edge

ITER 2 cov : `color_convert()` reached, `handle_8bit_alpha()` not taken

Diagnose: bottleneck = crafter; actual = wrong branch in right fn
Propose: (a) prompt *"start from real file, flip alpha bit-depth"* (b) add verifier + retry edge (c) wire san into verifier

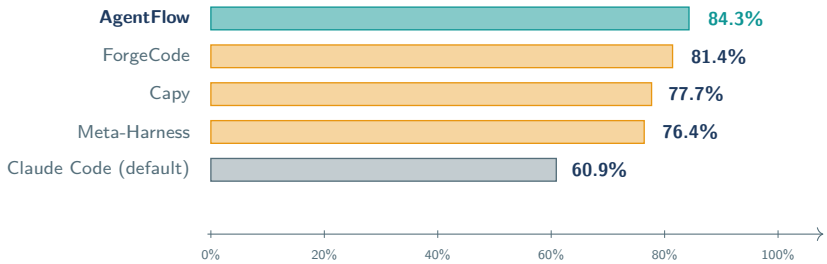
ITER 3 san fires: heap-buffer-overflow READ at `memcpy`

Score: incumbent updated. **CVE-2020-23109 reproduced.**

Same three iterations as slides 11–13 — **driven by the optimization loop**, not by a person.

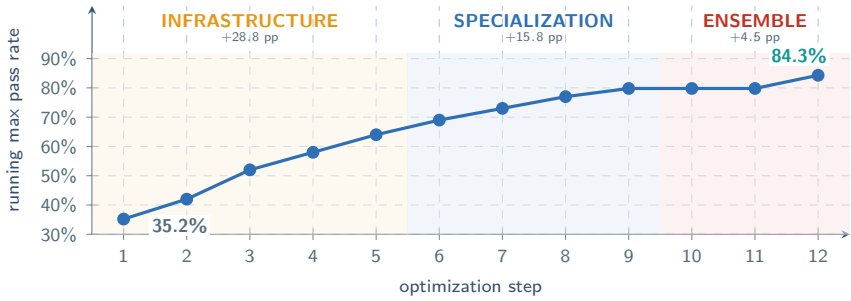
Results on TerminalBench-2

*89 long-horizon terminal tasks. **All entries: same Claude Opus 4.6** — the difference is the harness.*



+2.9 **pp** over best **hand-engineered** (ForgeCode) ·
+7.9 **pp** over best **synthesized** baseline (Meta-Harness)
Same model in every row. AgentFlow = **highest score** in the snapshot we evaluate against.

The Synthesis Trajectory



Across the campaign, edits touched **every one of** $\mathcal{A}$ $\mathcal{G}$ Σ Φ Ψ.
That is exactly what a DSL covering the **full search space** buys you.

Ablation: Both Insights Are Load-Bearing

*Two variants, each removes **one** contribution.*

AgentFlow (full system) **84.3%**

AgentFlow ★ — **NO TYPED DSL.** Skip the type check; proposer outputs go straight to dispatch.
⇒ malformed harnesses still fire LLMs ⇒ **consume search budget**. ↓ **drop**

AgentFlow ✠ — **NO RUNTIME FEEDBACK.** Replace the 4 channels with binary pass/fail.
⇒ diagnoser can't localize **why** ⇒ loop reduces to a **random walk**. ↓↓ **bigger drop**

Take away either contribution ⇒ AgentFlow is **no longer SoTA**. The two insights are **individually necessary**.

Real World: Ten Zero-Days in Google Chrome

Target: Google Chrome — **35M+ lines** of C/C++. Each subsystem exceeds any frontier context window.

Driver: **Kimi K2.5** (open-weight, mid-tier). *Same DSL, same loop, same type checker, same 4 channels.*

10 previously unknown ZERO-DAY VULNERABILITIES

all 10 confirmed by Google's Vulnerability Reward Program

2 CRITICAL sandbox-escape CVEs:

`CVE-2026-7363` `CVE-2026-6297`

attacker-controlled web page → code execution on the user's host

Synthesized harness: 7 subsystem analysts · attack-surface mapper · parallel explorers · 4-stage triage · 2-stage validation.

Why the Method Generalizes

TerminalBench-2

Model: Claude Opus 4.6
Domain: 89 short tasks
Scale: hours
Score: pass rate
Output: leaderboard rank

Google Chrome

Model: Kimi K2.5
Domain: 1 giant codebase
Scale: 7 days, 192 H100s
Score: unique crash signatures
Output: real CVEs

DIFFERENT: LLM model · domain · scale · score function

SAME: the DSL · the propose / observe / score / diagnose loop · the type checker · the 4 feedback channels

What transfers across the two campaigns is the **search procedure** — not the harness it produced.

Three Takeaways

① Harness design is the dominant lever.

Hold the model fixed and **only** changing the harness moves pass rates by **4×**.

If you build agent systems, your time is best spent on the harness — not on the model.

② A wide search needs a typed DSL.

Without typing, malformed candidates eat the budget. With typing, all five knobs ($\mathcal{A}$ $\mathcal{G}$ Σ Φ Ψ) become editable in **one program**.

The DSL is what makes the joint search tractable at all.

③ Rich feedback turns search into diagnosis.

Pass/fail tells you **that** something failed. **cov** **san** **stderr** tell you **which agent to fix**.

Anywhere you have richer-than-binary feedback, the same loop applies.

These ideas aren't tied to vulnerability discovery. Anywhere you have a multi-agent system + a richer-than-binary feedback channel, ***the same loop applies****.*

Thank you.

Synthesizing Multi-Agent Harnesses for Vulnerability Discovery

Hanzhi Liu Chaofan Shou Hongbo Wen
Yanju Chen Ryan Jingyang Fang Yu Feng

UC Santa Barbara · UC San Diego · Fuzzland · World Liberty Financial

arxiv.org/abs/2604.20801

Questions?